

Reviewer Pack — Minimal Local Induction Dimension in Boolean Algebra and Com...

Entry bf40d0b1b709 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 05:36:09 UTC

Minimal Local Induction Dimension in Boolean Algebra and Communication Complexity Rank Inequality

Entry ID: bf40d0b1b709 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 05:36:09 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Local Induction Dimension in Boolean Algebra) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every instance φ of a SAT formula with n variables, the local induction dimension (LID) of its negation normal form (NNF), $\text{LID}(\varphi_{\text{NNF}})$, is linearly correlated with the communication complexity rank (CCR) of φ , such that $\text{LID}(\varphi_{\text{NNF}}) = \Theta(\text{CCR}(\varphi))$.

Rationale (proposer's reasoning):

Local induction dimension in boolean algebra measures the complexity of a Boolean function by counting the minimum number of variables needed to define it. If this measure is related to communication complexity rank, it could provide a novel perspective on understanding the inherent complexity of SAT formulas and their resolution proof structures.

Taxonomy category: CommunicationComplexityRank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d948e4af0d3dac15

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\text{LID}(\varphi_{\text{NNF}})$ and $\text{CCR}(\varphi)$ exceeds 0.7 across all 30 seeds, and falsified if this coefficient falls below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local induction dimension" AND "Boolean algebra" AND "communication complexity" AND "matrix rank" - "negation normal form" IN BOOLEAN MODE AND "communication complexity" AND "rank inequality" AND "algebra" - "SAT formula" AND "local induction dimension" AND "communication complexity rank" AND "Boolean algebra"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_formula(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def nnf(sat_formula):
        n = int(math.log2(len(sat_formula)))
        nnf_formula = []
        for i in range(n):
            clause = []
            for j in range(i+1, n):
                if sat_formula[2**i+j] == 0:
                    clause.append(j)
            if clause:
                nnf_formula.append(clause)
        return nnf_formula

    def lid(nnf_formula):
        variables = set()
        for clause in nnf_formula:
            variables.update(clause)
        return len(variables)

    def ccr(sat_formula):
        n = int(math.log2(len(sat_formula)))
        matrix = [[0] * (n+1) for _ in range(n+1)]
        for i in range(2**n):
            row = [i >> j & 1 for j in range(n)]
```

```

        row.append(sat_formula[i])
        matrix[row[0]] = row
    rank = 0
    for i in range(n+1):
        if any(matrix[j][i] for j in range(i, n+1)):
            pivot_row = next(j for j in range(i, n+1) if matrix[j][i])
            matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
            rank += 1
            for j in range(n+1):
                if i != j:
                    factor = Fraction(matrix[j][i], matrix[i][i])
                    for k in range(n+1):
                        matrix[j][k] -= factor * matrix[i][k]
    return rank

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    var_x = sum((x[i] - mean_x)**2 for i in range(n)) / n
    var_y = sum((y[i] - mean_y)**2 for i in range(n)) / n
    return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

n_values = [5, 10, 15, 20, 30, 40]
lid_values = []
ccr_values = []

for n in n_values:
    sat_formula = generate_sat_formula(n)
    nnf_formula = nnf(sat_formula)
    lid_value = lid(nnf_formula)
    ccr_value = ccr(sat_formula)
    lid_values.append(lid_value)
    ccr_values.append(ccr_value)

correlation = pearson_correlation(lid_values, ccr_values)
conjecture_holds = 0.5 < correlation < 0.7
counterexample = "" if conjecture_holds else f"Correlation: {correlation}"

return {
    "metric_name": "Pearson Correlation",
    "metric_value": correlation,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='Correlation outside [0.5, 0.7]' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient statistical signal")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_591d1b6c.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_591d1b6c.py", line 77, in run_trial
    nnf_formula = nnf(sat_formula)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_591d1b6c.py", line 30, in nnf
    if sat_formula[2*(i+j)] == 0:
       ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient between $LID(\varphi_NNF)$ and $CCR(\varphi)$. As a result, we cannot confirm whether the conjecture is supported or falsified based on the pre-registered criteria.
 | next: Re-run the test with proper error handling to ensure it completes without crashing.
 If the test passes, re-evaluate the conjecture using the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13295
2	propose	ollama_remote	glm4:latest	0	0	13330
3	preregistration	ollama_remote	glm4:latest	0	0	9222
4	novelty	ollama_remote	glm4:latest	0	0	8695
5	novelty	ollama_remote	glm4:latest	0	0	8837
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15250
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16404
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40856
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23499
10	judge	ollama_remote	glm4:latest	0	0	76918

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 226307 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/bf40d0b1b709.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/bf40d0b1b709.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/bf40d0b1b709.tar.gz* (if generated)